The Journal of Open Source Software

QMCPy: An Open-Source Python Framework for (Quasi-)Monte Carlo Algorithms

Aleksei G. Sorokin [1], **Fred J. Hickernell** [1], **Sou-Cheng T. Choi** [1,2], **Jagadeeswaran Rathinavel** [3], **Pieterjan Robbe** [4], **and Aadit Jain** [5]

1 Illinois Institute of Technology, USA **2** SouLab LLC, USA **3** Torc Robotics, USA **4** Sandia National Laboratories, USA **5** University of California San Diego, USA

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 20 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

QMCPy is an open-source Python package for high-dimensional numerical integration using Monte Carlo (MC) and quasi-Monte Carlo (QMC) methods—collectively "(Q)MC." Its object-oriented (OO) design enables researchers to easily implement novel (Q)MC algorithms. The framework offers user-friendly APIs, diverse (Q)MC algorithms, adaptive error estimation, and integration with scientific libraries following reproducible research practices. Compared to previous versions, QMCPy v2.0 includes:

- improved documentation,
- strengthened tests and demos,
- expanded support for randomized low-discrepancy (LD) sequences,
- enhanced option pricing capabilities, and
- new utilities for fast QMC-based kernel methods.

Statement of Need

High-dimensional integration and simulation are essential for computational finance (Giles & Waterhouse, 2009; Lemieux, 2004; X. Wang & Sloan, 2005; Zhang et al., 2021), uncertainty quantification (Kaarnioja et al., 2021; Marzouk et al., 2016; Parno et al., 2014, 2021), machine learning (Chen et al., 2018; Dick & Feischl, 2021), and physics (Albert & Barabási, 2002; Bernhard et al., 2015; Landau & Binder, 2014). QMCPy (Choi, Hickernell, Rathinavel, et al., 2025b) implements both MC methods which use independent identically distributed (IID) sequences and QMC methods which use LD sequences that more evenly cover the unit cube, enabling faster convergence (Richtmyer, 1951). Figure 1 visualizes IID and LD sequences.

The Journal of Open Source Software

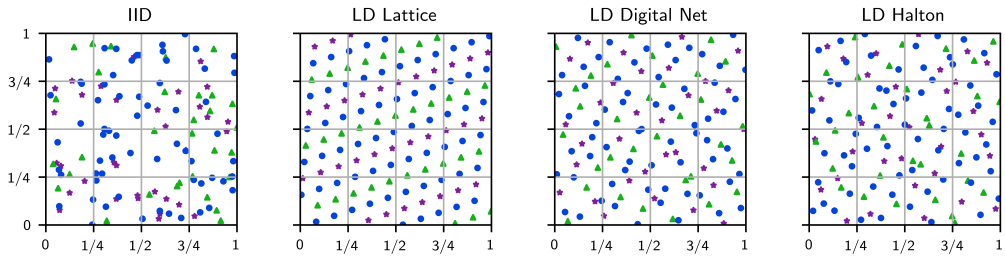

Figure 1: An IID sequence with gaps and clusters alongside LD sequences which more evenly fill the space. Each of the three randomized LD sequences contains purple stars (initial 32 points), green triangles (next 32), and blue circles (subsequent 64). The lattice was randomly shifted; the digital sequence was randomized with nested uniform scrambling (Owen scrambling); the Halton sequence was randomized with linear matrix scrambling and permutation scrambling.

While (Q)MC methods are well established (Dick et al., 2013; Dick & Pillichshammer, 2010b), practical implementation demands numerical and algorithmic expertise. Our `QMCPy` implementation follows MATLAB's Guaranteed Automatic Integration Library (GAIL) (Choi et al., 2021; Tong et al., 2022), with both adhering to reproducible research practices (Choi et al., 2022; Choi, 2014). However, `QMCPy` consolidates a substantially broader range of cutting-edge (Q)MC algorithms (Choi et al., 2022, 2024; Hickernell et al., 2025+; Sorokin, 2025a; Sorokin & Rathinavel, 2022) into a unified framework, bridging theory and practice, with the following features:

- **Intuitive APIs** for problem specification and accessing (Q)MC methods,
- **Flexible integrations** with `NumPy` (Harris et al., 2020) and `SciPy` (Virtanen et al., 2020),
- **Robust and adaptive sampling** with theoretically grounded error estimation, and
- **Extensible OO components** enabling researchers to implement and test new algorithms.

Unlike other (Q)MC modules such as `SciPy`'s `scipy.stats.qmc` (Virtanen et al., 2020) or `PyTorch`'s `torch.quasirandom` (Paszke et al., 2019), `QMCPy` provides:

- customizable LD sequences with diverse randomization techniques,
- automatic variable transformations for (Q)MC compatibility, and
- rigorous adaptive error estimation algorithms.

Components

(Q)MC methods approximate the multivariate integral

$$\mu := \mathbb{E}[g(\mathbf{T})] = \int_{\mathcal{T}} g(\mathbf{t})\,\lambda(\mathbf{t})\,d\mathbf{t}, \qquad \mathbf{T} \sim \lambda, \tag{1}$$

where g is the **integrand** and λ is the probability density of a random variable $\mathbf{T}$ whose distribution we call the **true measure**. Through an appropriate transformation ψ, we rewrite μ as the expectation of a function of a standard uniform random variable $\mathbf{X}$ over the unit hypercube:

$$\mu = \mathbb{E}[f(\mathbf{X})] = \int_{[0,1]^d} f(\mathbf{x})\,d\mathbf{x}, \qquad \mathbf{X} \sim \mathcal{U}[0,1]^d. \tag{2}$$

If ψ satisfies $\mathbf{T} \sim \psi(\mathbf{X})$, then $f = g \circ \psi$. This transformation accommodates IID and LD samples which are approximately uniform on $[0,1]^d$.

(Q)MC methods estimate the population mean μ in (2) via the sample mean

$$\hat{\mu} := \frac{1}{n}\sum_{i=1}^{n} f(\mathbf{X}_i). \tag{3}$$

DRAFT

MC methods choose IID sampling nodes $\mathbf{X}_1, \dots, \mathbf{X}_n$ and have error $|\hat{\mu} - \mu|$ like $\mathcal{O}(n^{-1/2})$ (Niederreiter, 1978). QMC methods choose dependent LD nodes that fill $[0,1]^d$ more evenly, achieving errors like $\mathcal{O}(n^{-1+\delta})$ where $\delta > 0$ is arbitrarily small (Hickernell & Wang, 2002; X. Wang, 2003). A key feature of `QMCPy` is **stopping criteria** that automatically determine n so $|\mu - \hat{\mu}| \le \varepsilon$ for a user-specified tolerance $\varepsilon > 0$, either deterministically or with high probability.

`QMCPy` contains four main abstract classes which are extensible for new (Q)MC algorithms:

1. **Discrete Distributions** generate IID or LD sequences. Available LD sequences and randomization routines (Sorokin, 2025a) include

   - **Lattices** with random shifts (Coveyou & MacPherson, 1967; Cranley & Patterson, 1976; Hickernell et al., 2005; Richtmyer, 1951; Y. Wang & Hickernell, 2002).
   - **Digital Sequences** (including Sobol' sequences) with digital shifts, linear matrix scrambling (LMS), or nested uniform scrambling (NUS, also called Owen scrambling) (Dick, 2011; Dick & Pillichshammer, 2005; Dick & Pillichshammer, 2010a; Matoušek, 1998; Niederreiter, 1987, 1992; Owen, 1995, 2003; Sobol', 1967).
   - **Halton Sequences** with digital shifts, permutation scrambling, LMS, or NUS (Halton, 1960; Matoušek, 1998; Morokoff & Caflisch, 1994; Owen & Pan, 2024, 2024; X. Wang & Hickernell, 2000).

   We support higher-order digital sequences and higher order scrambling for integrands f with α degrees of "smoothness", enabling QMC convergence like $\mathcal{O}(n^{-\alpha+\delta})$ where $\delta > 0$ is arbitrarily small (Dick, 2011). Internally, our LD generators call our C package `QMCToolsCL` (Sorokin, 2025b). `QMCPy` also integrates with the `LDData` repository (Sorokin, Hickernell, & L'Ecuyer, 2025) which stores generating vectors for lattices and generating matrices for digital sequences, primarily drawing from the Magic Point Shop (Kuo & Nuyens, 2016) and Kuo's websites on lattices (Cools et al., 2006; Kuo, 2007; Nuyens & Cools, 2006) and Sobol' sequences (Joe & Kuo, 2003, 2010; Joe & Kuo, 2008).

2. **True Measures** come with default transformations ψ satisfying $\psi(\mathbf{X}) \sim \mathbf{T}$ where $\mathbf{X} \sim \mathcal{U}[0,1]^d$. For example, suppose $\mathbf{T} \sim \mathcal{N}(\mathbf{m}, \Sigma)$ is a d-dimensional Gaussian random variable with mean $\mathbf{m}$ and covariance $\Sigma = \mathbf{A}\mathbf{A}^T$. Then $\psi(\mathbf{X}) = \mathbf{A}\Phi^{-1}(\mathbf{X}) + \mathbf{m}$ where Φ^{-1} is the inverse distribution function of a standard normal applied elementwise. We support many measures, including those from `SciPy`'s `scipy.stats` (Virtanen et al., 2020).

3. **Integrands** g, given a transformation ψ, define the transformed integrand $f = g \circ \psi$ so that $\mu = \mathbb{E}[g(\mathbf{T})] = \mathbb{E}[f(\mathbf{X})]$ where $\mathbf{X} \sim \mathcal{U}[0,1]^d$. This change of variables is performed automatically. Users only need to specify g and $\mathbf{T}$.

4. **Stopping Criteria (SC)** adaptively increase the sample size n until (Q)MC estimates satisfy user-defined error tolerances (Hickernell, Choi, et al., 2018; Owen, 2024; Tong et al., 2022). SC vary depending on properties of f, and can include guaranteed MC algorithms (Hickernell et al., 2013) or QMC algorithms based on:

   - multiple randomizations of LD sequences (L'Ecuyer et al., 2023),
   - quickly tracking the decay of Fourier coefficients (Ding et al., 2020; Hickernell, Jiménez Rugama, et al., 2018; Hickernell & Jiménez Rugama, 2016; Jiménez Rugama & Hickernell, 2016), or
   - efficient Bayesian cubature by inducing structured Gram matrices (Rathinavel, 2019; Rathinavel & Hickernell, 2019, 2022).

   `QMCPy` is also capable of simultaneously approximating functions of multiple integrands (Sorokin & Rathinavel, 2022). Inspired by `MultilevelEstimators.jl` (Robbe, 2024), `QMCPy` is expanding support for multilevel (Q)MC SC (Giles, 2015; Giles & Waterhouse, 2009) that exploit cheaper, low-fidelity surrogates to accelerate expensive integrand estimates in high or infinite dimensions.

DRAFT

Figure 2 compares MC and QMC SC performance for adaptively estimating the fair price of an Asian option across 100 trials per error tolerance. Both methods consistently meet tolerances. The left panel shows sample complexity: MC algorithms require $n = \mathcal{O}(1/\varepsilon^2)$ samples while QMC algorithms require only $n = \mathcal{O}(1/\varepsilon)$ samples, with shaded regions showing 10%-90% quantiles. The middle panel displays the computation time, highlighting that QMC methods are much faster than MC methods. The right panel presents error distributions via violin plots for a single error tolerance, showing that the average error performance is better for QMC methods.

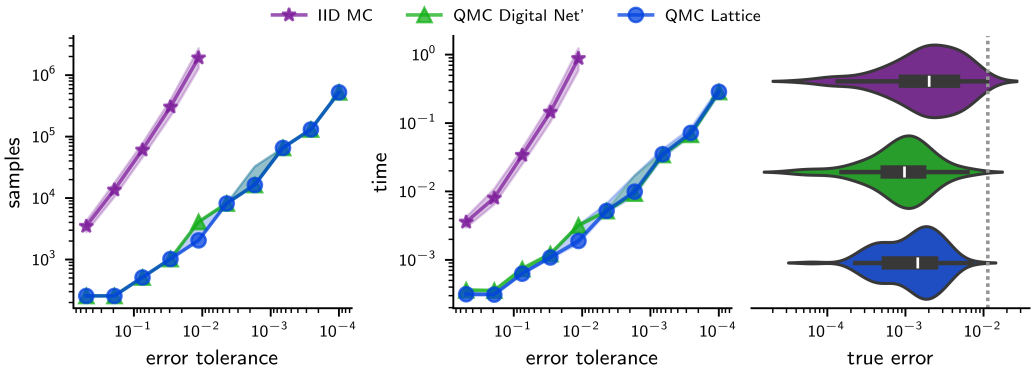

Figure 2: MC and QMC SC comparison for pricing an Asian option.

Distribution and Resources

`QMCPy` can be installed using the command `pip install qmcpy` (Choi, Hickernell, Jagadeeswaran, et al., 2025). Figure 1 and Figure 2 are reproducible via the Jupyter Notebook (Sorokin, Hickernell, Choi, et al., 2025). Our project website (Choi, Hickernell, Rathinavel, et al., 2025a) features publications, presentations, blogs, documentation (Choi et al., 2020), and demos. Our GitHub repository (Choi, Hickernell, McCourt, et al., 2025) contains open-source code with continuous integration, tests, and issue tracking (Choi et al., 2022; Sorokin, 2025a). `QMCPy` is distributed under the Apache (v2.0) license. Community feedback and engagement are welcome.

Acknowledgements

The authors acknowledge support from the U.S. National Science Foundation grant DMS-231601 and Department of Energy Office of Science Graduate Student Research Program. We thank the international (Q)MC research community for invaluable feedback and support.

This article has been co-authored by employees of National Technology and Engineering Solutions of Sandia, LLC under Contract No. DE-NA0003525 with the U.S. Department of Energy (DOE). The employees co-own right, title and interest in and to the article and are responsible for its contents. The United States Government retains and the publisher, by accepting the article for publication, acknowledges that the United States Government retains a non-exclusive, paid-up, irrevocable, world-wide license to publish or reproduce the published form of this article or allow others to do so, for United States Government purposes. The DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (https://www.energy.gov/downloads/doe-public-access-plan).

DRAFT

References

Albert, R., & Barabási, A.-L. (2002). Statistical mechanics of complex networks. *Rev. Modern Phys.*, *74*(1), 47–97.

Bernhard, J. E., Marcy, P. W., Coleman-Smith, C. E., Huzurbazar, S., Wolpert, R. L., & Bass, S. A. (2015). Quantifying properties of hot and dense QCD matter through systematic model-to-data comparison. *Physical Review C*, *91*(5), 054910.

Chen, W. Y., Mackey, L., Gorham, J., Briol, F.-X., & Oates, C. (2018). Stein points. In J. Dy & A. Krause (Eds.), *Proceedings of the 35th international conference on machine learning* (Vol. 80, pp. 844–853). PMLR. https://proceedings.mlr.press/v80/chen18f.html

Choi, S.-C. T. (2014). MINRES-QLP Pack and reliable reproducible research via staunch scientific software. *Journal of Open Research Software*, *2*(1), 1–7. https://doi.org/10.5334/jors.bb

Choi, S.-C. T., Ding, Y., Hickernell, F. J., Jiang, L., Jiménez Rugama, Ll. A., Li, D., Rathinavel, J., Tong, X., Zhang, K., Zhang, Y., & Zhou, X. (2021). *GAIL: Guaranteed Automatic Integration Library (versions 1.0–2.3.2)*. MATLAB software, http://gailgithub.github.io/GAIL_Dev/. https://doi.org/10.5281/zenodo.4018189

Choi, S.-C. T., Ding, Y., Hickernell, F. J., Rathinavel, J., & Sorokin, A. G. (2024). Challenges in developing great quasi-Monte Carlo software. In A. Hinrichs, P. Kritzer, & F. Pillichshammer (Eds.), *Monte Carlo and quasi-Monte Carlo methods: MCQMC, Linz, austria, July 2022* (pp. 209–222). Springer, Cham. https://doi.org/10.1007/978-3-031-59762-6/_9

Choi, S.-C. T., Hickernell, F. J., Jagadeeswaran, R., McCourt, M., & Sorokin, A. G. (2025). *QMCPy: Quasi-Monte Carlo Community Software in Python*. https://pypi.org/project/qmcpy/.

Choi, S.-C. T., Hickernell, F. J., McCourt, M., Rathinavel, J., & Sorokin, A. G. (2025). *QMCPy: Quasi-Monte Carlo community software. Python 3 software.* GitHub. https://github.com/QMCSoftware/QMCSoftware

Choi, S.-C. T., Hickernell, F. J., Rathinavel, J., McCourt, M. J., & Sorokin, A. (2020). *QMCPy documentation*. https://qmcsoftware.github.io/QMCSoftware/

Choi, S.-C. T., Hickernell, F. J., Rathinavel, J., McCourt, M. J., & Sorokin, A. (2025a). *QMCPy blogs*. https://qmcpy.org

Choi, S.-C. T., Hickernell, F. J., Rathinavel, J., McCourt, M. J., & Sorokin, A. G. (2022). Quasi-Monte Carlo software. In A. Keller (Ed.), *Monte Carlo and quasi-Monte Carlo methods: MCQMC, Oxford, england, August 2020* (pp. 23–50). Springer, Cham.

Choi, S.-C. T., Hickernell, F. J., Rathinavel, J., McCourt, M., & Sorokin, A. G. (2025b). *QMCPy: A quasi-Monte Carlo Python library* (Version 1.0–2.0). Python Software, Zenodo. https://doi.org/10.5281/zenodo.3964489

Cools, R., Kuo, F. Y., & Nuyens, D. (2006). Constructing embedded lattice rules for multivariate integration. *SIAM Journal on Scientific Computing*, *28*(6), 2162–2188.

Coveyou, R., & MacPherson, R. D. (1967). Fourier analysis of uniform random number generators. *Journal of the ACM (JACM)*, *14*(1), 100–119.

Cranley, R., & Patterson, T. N. L. (1976). Randomization of number theoretic methods for multiple integration. *SIAM J. Numer. Anal.*, *13*, 904–914.

Dick, J. (2011). Higher order scrambled digital nets achieve the optimal rate of the root mean square error for smooth integrands. *The Annals of Statistics*, *39*(3), 1372–1398. https://doi.org/10.1214/11-AOS880

DRAFT

Dick, J., & Feischl, M. (2021). A quasi-Monte Carlo data compression algorithm for machine learning. *J. Complexity*, *67*, 101587. https://doi.org/https://doi.org/10.1016/j.jco.2021.101587

Dick, J., Kuo, F. Y., & Sloan, I. H. (2013). High-dimensional integration: The quasi-Monte Carlo way. *Acta Numerica*, *22*, 133–288.

Dick, J., & Pillichshammer, F. (2005). Multivariate integration in weighted Hilbert spaces based on Walsh functions and weighted Sobolev spaces. *Journal of Complexity*, *21*(2), 149–195.

Dick, J., & Pillichshammer, F. (2010a). *Digital nets and sequences: Discrepancy theory and quasi-Monte Carlo integration*. Cambridge University Press.

Dick, J., & Pillichshammer, F. (2010b). *Digital nets and sequences: Discrepancy theory and quasi–Monte Carlo integration*. Cambridge University Press.

Ding, Y., Hickernell, F. J., & Jiménez Rugama, Ll. A. (2020). An adaptive algorithm employing continuous linear functionals. In P. L'Ecuyer & B. Tuffin (Eds.), *Monte Carlo and quasi-Monte Carlo methods: MCQMC, Rennes, france, July 2018* (Vol. 324, pp. 161–181). Springer, Cham. https://doi.org/10.1007/978-3-030-43465-6/_8

Giles, M. B. (2015). Multilevel Monte Carlo methods. *Acta Numerica*, *24*, 259–328.

Giles, M. B., & Waterhouse, B. J. (2009). Multilevel quasi-Monte Carlo path simulation. *Advanced Financial Modelling, Radon Series on Computational and Applied Mathematics*, *8*, 165–181.

Halton, J. H. (1960). On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numer. Math.*, *2*, 84–90.

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hickernell, F. J., Choi, S.-C. T., Jiang, L., & Jiménez Rugama, Ll. A. (2018). Monte Carlo simulation, automatic stopping criteria for. In M. Davidian, B. Everitt, R. S. Kenett, G. Molenberghs, W. Piegorsch, & F. Ruggeri (Eds.), *Wiley StatsRef-statistics reference online*. John Wiley & Sons Ltd. https://doi.org/10.1002/9781118445112.stat08035

Hickernell, F. J., Jiang, L., Liu, Y., & Owen, A. B. (2013). Guaranteed conservative fixed width confidence intervals via Monte Carlo sampling. In J. Dick, F. Y. Kuo, G. W. Peters, & I. H. Sloan (Eds.), *Monte Carlo and quasi-Monte Carlo methods 2012* (Vol. 65, pp. 105–128). Springer-Verlag, Berlin. https://doi.org/10.1007/978-3-642-41095-6

Hickernell, F. J., & Jiménez Rugama, Ll. A. (2016). Reliable adaptive cubature using digital sequences. In R. Cools & D. Nuyens (Eds.), *Monte Carlo and quasi-Monte Carlo methods: MCQMC, Leuven, Belgium, April 2014* (Vol. 163, pp. 367–383). Springer-Verlag, Berlin.

Hickernell, F. J., Jiménez Rugama, Ll. A., & Li, D. (2018). Adaptive quasi-Monte Carlo methods for cubature. In J. Dick, F. Y. Kuo, & H. Woźniakowski (Eds.), *Contemporary computational mathematics — a celebration of the 80th birthday of Ian Sloan* (pp. 597–619). Springer-Verlag. https://doi.org/10.1007/978-3-319-72456-0

Hickernell, F. J., Kirk, N., & Sorokin, A. G. (2025+). Quasi-Monte Carlo methods: What, why, and how? In C. Lemieux & B. Feng (Eds.), *Monte Carlo and quasi-Monte Carlo methods: MCQMC, Waterloo, Canada, August 2024*. Accepted for publication; Springer, Cham. https://doi.org/10.48550/arXiv.2502.03644

Hickernell, F. J., Lemieux, C., & Owen, A. B. (2005). Control variates for quasi-Monte Carlo.

DRAFT

Statist. Sci., *20*, 1–31. https://doi.org/10.1214/088342304000000468

Hickernell, F. J., & Wang, X. (2002). The error bounds and tractability of quasi-Monte Carlo algorithms in infinite dimension. *Math. Comp.*, *71*, 1641–1661. https://doi.org/10.1090/S0025-5718-01-01377-1

Jiménez Rugama, Ll. A., & Hickernell, F. J. (2016). Adaptive multidimensional integration based on rank-1 lattices. In R. Cools & D. Nuyens (Eds.), *Monte Carlo and quasi-Monte Carlo methods: MCQMC, Leuven, Belgium, April 2014* (Vol. 163, pp. 407–422). Springer-Verlag, Berlin.

Joe, S., & Kuo, F. Y. (2003). Remark on algorithm 659: Implementing Sobol's quasirandom sequence generator. *ACM Trans. Math. Software*, *29*, 49–57.

Joe, S., & Kuo, F. Y. (2008). Constructing Sobol sequences with better two-dimensional projections. *SIAM Journal on Scientific Computing*, *30*(5), 2635–2654.

Joe, S., & Kuo, F. Y. (2010). *Sobol sequence generator*. https://web.maths.unsw.edu.au/~fkuo/sobol/

Kaarnioja, V., Kazashi, Y., Kuo, F. Y., Nobile, F., & Sloan, I. H. (2021). Fast approximation by periodic kernel-based lattice-point interpolation with application in uncertainty quantification. *Numer. Math.*, *150*, 33–77.

Kuo, F. Y. (2007). *Lattice rule generating vectors*. https://web.maths.unsw.edu.au/~fkuo/lattice/

Kuo, F. Y., & Nuyens, D. (2016). Application of quasi-Monte Carlo methods to elliptic PDEs with random diffusion coefficients – a survey of analysis and implementation. *Found. Comput. Math.*, *16*, 1631–1696. https://people.cs.kuleuven.be/~dirk.nuyens/qmc4pde/

L'Ecuyer, P., Nakayama, M. K., Owen, A. B., & Tuffin, B. (2023). Confidence intervals for randomized quasi-Monte Carlo estimators. *2023 Winter Simulation Conference (WSC)*, 445–456.

Landau, D. P., & Binder, K. (2014). *A guide to Monte Carlo simulations in statistical physics*. Cambridge University Press.

Lemieux, C. (2004). Randomized quasi-Monte Carlo: A tool for improving the efficiency of simulations in finance. In R. G. Ingalls, M. D. Rossetti, J. S. Smith, & B. A. Peters (Eds.), *Proc. 2004 winter simulation conference* (pp. 1565–1573). IEEE Press.

Marzouk, Y., Moselhy, T., Parno, M., & Spantini, A. (2016). Sampling via measure transport: An introduction. In R. Ghanem, D. Higdon, & H. Owhadi (Eds.), *Handbook of uncertainty quantification* (pp. 1–41). Springer International Publishing. https://doi.org/10.1007/978-3-319-11259-6/_23-1

Matoušek, J. (1998). On the L_2-discrepancy for anchored boxes. *J. Complexity*, *14*, 527–556.

Morokoff, W. J., & Caflisch, R. E. (1994). Quasi-random sequences and their discrepancies. *SIAM J. Sci. Comput.*, *15*, 1251–1279.

Niederreiter, H. (1978). Quasi-Monte Carlo methods and pseudo-random numbers. *Bull. Amer. Math. Soc.*, *84*, 957–1041.

Niederreiter, H. (1987). Point sets and sequences with small discrepancy. *Monatsh. Math.*, *104*, 273–337.

Niederreiter, H. (1992). *Random number generation and quasi-Monte Carlo methods*. SIAM.

Nuyens, D., & Cools, R. (2006). Fast algorithms for component-by-component construction of rank-1 lattice rules in shift-invariant reproducing kernel Hilbert spaces. *Mathematics of Computation*, *75*(254), 903–920.

DRAFT

Owen, A. B. (1995). Randomly permuted (t, m, s)-nets and (t, s)-sequences. In H. Niederreiter & P. J.-S. Shiue (Eds.), *Monte Carlo and quasi-Monte Carlo methods in scientific computing* (Vol. 106, pp. 299–317). Springer-Verlag, New York.

Owen, A. B. (2003). Variance and discrepancy with alternative scramblings. *ACM Transactions of Modeling and Computer Simulation*, *13*(4).

Owen, A. B. (2024). Error estimation for quasi-Monte Carlo. *arXiv Preprint arXiv:2501.00150*.

Owen, A. B., & Pan, Z. (2024). Gain coefficients for scrambled Halton points. *SIAM Journal on Numerical Analysis*, *62*(3), 1021–1038.

Parno, M., Davis, A., & Seelinger, L. (2021). MUQ: The MIT Uncertainty Quantification library. *Journal of Open Source Software*, *6*(68), 3076.

Parno, M., Davis, A., Seelinger, L., & Marzouk, Y. (2014). *MIT uncertainty quantification (MUQ) library*. https://mituq.bitbucket.io/source/_site/index.html

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems 32* (pp. 8024–8035). Curran Associates, Inc. http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

Rathinavel, J. (2019). *Fast automatic Bayesian cubature using matching kernels and designs* [PhD thesis]. Illinois Institute of Technology.

Rathinavel, J., & Hickernell, F. J. (2019). Fast automatic Bayesian cubature using lattice sampling. *Stat. Comput.*, *29*, 1215–1229. https://doi.org/10.1007/s11222-019-09895-9

Rathinavel, J., & Hickernell, F. J. (2022). Fast automatic Bayesian cubature using Sobol' sampling. In Z. Botev, A. Keller, C. Lemieux, & B. Tuffin (Eds.), *Advances in modeling and simulation: Festschrift in honour of Pierre L'Ecuyer* (pp. 301–318). Springer, Cham. https://doi.org/10.1007/978-3-031-10193-9/_15

Richtmyer, R. D. (1951). *The evaluation of definite integrals, and a Quasi-Monte-Carlo method based on the properties of algebraic numbers* (LA-1342). Los Alamos Scientific Laboratory.

Robbe, P. (2024). *MultilevelEstimators.jl: A Julia package for multilevel Monte Carlo*. https://github.com/PieterjanRobbe/MultilevelEstimators.jl

Sobol', I. M. (1967). The distribution of points in a cube and the approximate evaluation of integrals. *U.S.S.R. Comput. Math. And Math. Phys.*, *7*, 86–112.

Sorokin, A. G. (2025a). QMCPy: A Python software for randomized low-discrepancy sequences, quasi-Monte Carlo, and fast kernel methods. *arXiv Preprint arXiv:2502.14256*.

Sorokin, A. G. (2025b). *QMCToolsCL: Quasi-Monte Carlo sequence generators in OpenCL and C*. https://github.com/QMCSoftware/QMCToolsCL.

Sorokin, A. G., Hickernell, F. J., Choi, S.-C. T., Jain, A., Rathinavel, J., & Robbe, P. (2025). *QMCPy JOSS2025: Reproducible figures and demos*. https://github.com/QMCSoftware/QMCSoftware/blob/joss/demos/talk_paper_demos/JOSS2025/joss2025.ipynb.

Sorokin, A. G., Hickernell, F. J., & L'Ecuyer, P. (2025). *LDData: Low discrepancy generating vectors and matrices*. https://github.com/QMCSoftware/LDData.

Sorokin, A. G., & Rathinavel, J. (2022). On bounding and approximating functions of multiple expectations using quasi-Monte Carlo. *International Conference on Monte Carlo and Quasi-Monte Carlo Methods in Scientific Computing*, 583–599.

Tong, X., Choi, S.-C. T., Ding, Y., Hickernell, F. J., Jiang, L., Jiménez Rugama, Ll. A., Rathinavel, J., Zhang, K., Zhang, Y., & Zhou, X. (2022). Guaranteed automatic integration

DRAFT

library (GAIL): An open-source MATLAB library for function approximation, minimization, and integration. *Journal of Open Research Software*, *10*, 7. https://doi.org/10.5334/jors.381

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wang, X. (2003). Strong tractability of multivariate integration using Quasi-Monte Carlo algorithms. *Math. Comp.*, *72*, 823–838.

Wang, X., & Hickernell, F. J. (2000). Randomized Halton sequences. *Math. Comput. Modelling*, *32*, 887–899.

Wang, X., & Sloan, I. H. (2005). Why are high-dimensional finance problems often of low effective dimension? *SIAM J. Sci. Comput.*, *27 (1)*, 159–183.

Wang, Y., & Hickernell, F. J. (2002). An historical overview of lattice point sets. In K. T. Fang, F. J. Hickernell, & H. Niederreiter (Eds.), *Monte Carlo and quasi-Monte Carlo methods 2000* (pp. 158–167). Springer-Verlag, Berlin.

Zhang, Y., Li, J., Wang, H., & Choi, S.-C. T. (2021). Sentiment-guided adversarial learning for stock price prediction. *Frontiers in Applied Mathematics and Statistics*, *7*, 8.

DRAFT